PERSONAL PROJECT REPORT

FACE RECOGNITION BASED ATTENDANCE MANAGEMENT SYSTEM

A practical Python and OpenCV application for detecting registered faces and recording attendance

Prepared by

P. THARUN SAI RAM

Project documentation • Design • Implementation • Testing • Results

Personal learning and development record

TABLE OF CONTENTS

01 Introduction and project context
02 Problem definition and objectives
03 Scope, assumptions and constraints
04 Background: face recognition pipeline
05 Technology stack and development environment
06 System requirements
07 System architecture and data flow
08 Dataset organization and preparation
09 Image acquisition and preprocessing
10 Face detection using Haar Cascade
11 Face recognition using LBPH
12 Model training workflow
13 Attendance recording and CSV design
14 Duplicate prevention logic
15 Application workflow and user operation
16 Implementation structure and key code patterns
17 Installation and setup guide
18 Git and project version control
19 Testing strategy and test cases
20 Observed demonstration results
21 Performance considerations
22 Security, privacy and responsible use
23 Limitations and risk analysis
24 Troubleshooting guide
25 Future enhancements
26 Conclusion and learning outcomes
27 Appendix A: project folder and commands
28 Appendix B: illustrative code snippets
29 Appendix C: glossary and references

1. INTRODUCTION

Attendance management is a routine process in schools, colleges, offices, laboratories and training environments. A conventional register or spreadsheet requires a person to identify each participant and record their presence. This can consume time, especially when many people arrive together.

Face recognition offers one possible way to automate part of this process. A camera captures a face image; software detects the face region and compares it with examples enrolled earlier. If the system finds a sufficiently close match, it can create an attendance entry.

This project is a learning-oriented prototype. It demonstrates how computer vision libraries can be connected to a practical record-keeping task. It is not intended to replace supervised attendance procedures without further validation, security controls and consent.

Project motivation

The main motivation was to build a working end-to-end application rather than only study isolated computer-vision functions. The project links image capture, detection, recognition, file handling and duplicate checking into one workflow.

2. PROBLEM DEFINITION AND OBJECTIVES

The problem addressed is the repetitive manual effort involved in taking attendance and maintaining a time-stamped record. Manual entry may be slow and can contain spelling mistakes, missed entries or duplicate entries. A face-based prototype can reduce some of this repetitive work when the camera view and enrolled images are suitable.

Project objectives are to capture live webcam frames, detect visible faces, train a recognizer from locally organized images, identify a known person, record the attendance date and time, and avoid repeated records for that person on the same day.

The project also aims to provide practical experience with Python environments, package installation, OpenCV APIs, CSV file operations, Git commits and a public source-code repository.

Success criteria

The application starts without a missing-dependency error; model training completes; the webcam opens; a registered face can be recognized in the demonstration conditions; an attendance row is written; and a second run on the same day does not create a duplicate row.

3. SCOPE, ASSUMPTIONS AND CONSTRAINTS

The current prototype supports local face images arranged in person-specific folders and a webcam-based attendance session. It writes a simple CSV record with the fields Name, Date and Time. The duplicate rule is based on a person's name and calendar date.

The implementation assumes that each registered person's images are stored in a folder named for that person under known_faces. It assumes the camera is available to the operating system and that the user has permission to access it. It also assumes the machine's date and time are reasonably correct.

Out of scope are cloud synchronization, multi-camera support, a mobile application, centralized user management, encrypted biometric storage, liveness detection, anti-spoofing, role-based administration and formal accuracy certification. The prototype should be used only in a controlled learning or demonstration setting.

Constraints

Recognition quality can change with lighting, pose, camera quality, distance, image resolution and the number and variety of enrollment images. A printed photograph or screen image may fool a system without liveness checks.

4. BACKGROUND: FACE RECOGNITION PIPELINE

A face-recognition system commonly contains several stages. First, image acquisition obtains a frame from a camera or file. Second, face detection locates one or more face regions. Third, preprocessing converts the detected region into a consistent representation. Fourth, feature extraction or a recognition model compares the face against enrolled examples. Finally, an application layer uses the result to perform an action.

Detection and recognition are different tasks. Detection answers where a face is in an image. Recognition estimates whether the detected face corresponds to a known enrolled identity. A detector can locate a face even when the person is not registered; the recognizer then decides whether it matches a trained identity.

In this project, Haar Cascade is used for detection and Local Binary Patterns Histograms (LBPH) is used for recognition. LBPH is a classical computer-vision approach that can be useful for small demonstrations, but it is not equivalent to modern deep-learning face embeddings.

Pipeline summary

Camera frame → grayscale conversion → face detection → face crop/normalization → LBPH prediction → confidence/threshold decision → attendance file update.

5. TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

Python is the main programming language. It provides a readable way to connect the camera, image-processing functions, model training and file operations. OpenCV supplies camera access, image conversion, Haar Cascade detection and the face module.

NumPy supports array operations used by image-processing libraries. Flask, pandas, openpyxl and Pillow were included in the project dependencies for possible extension and data handling; the core webcam attendance workflow does not require every listed package to be active at runtime.

The project was developed in Visual Studio Code on Windows. Python 3.12 was selected for the project virtual environment, while other Python versions may also be installed on the computer. Git was used to track source changes and publish the code repository.

Environment choices

A virtual environment isolates project packages from system-wide Python installations. This reduces dependency conflicts and makes the project easier to reproduce on another computer.

6. SYSTEM REQUIREMENTS

Hardware requirements include a Windows laptop or desktop, a working built-in or USB webcam, sufficient free disk space for Python packages and enrollment images, and a stable power supply during testing. A dedicated graphics processor is not required for this classical CPU-based prototype.

Software requirements include Windows, Python 3.12, Visual Studio Code or another editor, Git for version control, and the Python packages listed in requirements.txt. Camera permission must be enabled in Windows privacy settings.

Recommended operational conditions are a well-lit room, a camera positioned near eye level, a neutral background where possible, and a face positioned at a moderate distance from the camera. These are practical recommendations rather than formal performance guarantees.

Minimum practical checklist

- Python launcher can locate Python 3.12.
- The virtual environment is activated before installing or running dependencies.
- OpenCV imports successfully and exposes cv2.face.LBPHFaceRecognizer_create.
- The webcam is not being used exclusively by another application.
- The project directory is writable so attendance.csv can be updated.

7. SYSTEM ARCHITECTURE AND DATA FLOW

The application can be understood as five cooperating components: enrollment data, model training, camera capture, recognition decision and attendance storage. Enrollment data is held in folders. Training reads those images and builds an LBPH model. The camera component supplies live frames. The recognition component predicts a label for each detected face. The attendance component writes a row when the identity is accepted.

The architecture is intentionally local. Face images and attendance records are read from and written to the project directory. No remote database or hosted recognition service is required for the demonstrated workflow.

The data flow is sequential: the model is prepared from the known-face dataset before the live attendance loop begins. The live loop repeatedly captures frames, detects faces and evaluates them. A successful identity is passed to the attendance function, which checks whether a same-day record already exists.

Logical components

Enrollment folder → trainer → trained recognizer → webcam frame processor → identity decision → duplicate checker → attendance.csv.

8. DATASET ORGANIZATION AND PREPARATION

The dataset is organized by identity. The project uses a parent folder named known_faces, with a separate subfolder for each registered person. The demonstrated user's folder is known_faces/Tharun. The trainer can use folder names as identity labels, provided the folder structure is consistent.

Images should be genuine enrollment samples collected with the person's knowledge and permission. A useful small dataset includes modest variations in head angle, facial expression, distance and lighting. Images should be clear, contain one main face and avoid heavy blur or obstruction.

File extensions should be valid image formats supported by OpenCV. Inconsistent naming is usually acceptable if the image loader checks the extension rather than expecting one exact filename. Empty folders should not be treated as enrolled identities.

Dataset quality

A larger count alone does not guarantee better recognition. Diversity and image quality matter. Avoid collecting unnecessary images and never publish biometric photographs in a public source-code repository.

9. IMAGE ACQUISITION AND PREPROCESSING

OpenCV's VideoCapture interface provides access to a camera. The program requests a frame repeatedly and checks whether the capture succeeded. A failed frame should not be passed to image-processing functions. The camera resource should be released when the user exits the application.

Preprocessing commonly converts a color frame to grayscale because Haar Cascade and the LBPH workflow operate on intensity patterns. A face detector returns rectangular coordinates. The application can crop the face region and resize or normalize it to a consistent size before recognition.

Preprocessing should be applied consistently during both training and live prediction. If training images and webcam crops differ greatly in scale, orientation or preprocessing, recognition may become less reliable.

Typical operations

Capture frame; validate frame; convert BGR to grayscale; detect face rectangles; crop the detected region; resize if required; pass the crop to the recognizer.

10. FACE DETECTION USING HAAR CASCADE

Haar Cascade is a classical object-detection method based on Haar-like features and a cascade of classifiers. A cascade evaluates image regions through a sequence of stages. Regions that fail early stages are rejected quickly; regions that pass all stages are returned as possible faces.

The project loads the frontal-face XML classifier file haarcascade_frontalface_default.xml. The classifier file must exist at the path used by the application. During setup, the file was downloaded into the project root and the code was configured to load it from that location.

The detector is generally applied to a grayscale frame. Parameters such as scaleFactor, minNeighbors and minSize influence detection behavior. These values trade off sensitivity and false detections. The appropriate values depend on camera distance, image size and lighting.

Important distinction

A Haar Cascade result only indicates a likely face location. It does not identify the person. Recognition is performed by the LBPH model in a later stage.

11. FACE RECOGNITION USING LBPH

Local Binary Patterns describe local texture by comparing a pixel with neighboring pixels and encoding the comparison pattern. A histogram of these patterns summarizes texture information in a face image. LBPH recognition compares the histogram representation of an input face with representations learned from training images.

OpenCV's face module provides an LBPH face recognizer. The project uses the contrib build of OpenCV because the face module is not included in every OpenCV package distribution. The environment check confirmed that cv2.face.LBPHFaceRecognizer_create was available.

LBPH prediction returns a predicted label and a distance-like confidence value. In this context, a smaller distance generally indicates a closer match, but the threshold must be selected and validated for the particular dataset. A threshold should not be presented as a universal accuracy score.

Caution about confidence

The model's distance value is not a calibrated probability. It should be tuned using separate known and unknown test images. The current demonstration confirms basic recognition, not a scientifically measured false-accept or false-reject rate.

12. MODEL TRAINING WORKFLOW

Training begins by scanning the known_faces directory. Each person's subfolder provides the identity label. The trainer reads supported image files, converts them to grayscale, detects faces or extracts suitable face regions, and associates each usable sample with the folder's label.

The collected face samples and numeric labels are passed to the LBPH recognizer's training method. If no usable face samples are found, training should stop with a clear message rather than launching a live camera session with an empty model.

The demonstrated application printed a successful model-training message before starting the attendance workflow. For repeatable development, the training stage should be run whenever enrollment images are added, removed or replaced.

Training checks

- Confirm each identity folder contains readable images.
- Check that face detection succeeds on enough samples.
- Ensure labels map back to the correct folder names.
- Handle empty or unreadable images safely.
- Retest recognition after changing the dataset or threshold.

13. ATTENDANCE RECORDING AND CSV DESIGN

The attendance file is a comma-separated values document named attendance.csv. It is human-readable and can be opened in a text editor or spreadsheet application. The demonstrated schema contains three columns: Name, Date and Time.

Each successful attendance event is represented by one row. The name identifies the recognized person, the date records the local calendar date and the time records the local clock time at which the event was accepted.

A CSV file is suitable for a small single-machine prototype. It is not a transactional database. Concurrent writes, manual edits, file corruption, timezone handling and long-term audit requirements would need additional engineering in a production system.

Example record format

- **Name,Date,Time**
- **Tharun,2026-11-10,12:49:11**
- Example only: the row above illustrates the file format and is not a claim about every future run.

14. DUPLICATE PREVENTION LOGIC

The project checks existing attendance entries before writing a new row. The intended rule is one attendance record per recognized person per calendar day. The check compares both the person's name and the date.

This prevents repeated detections during a continuous webcam session from producing a long sequence of identical entries. The demonstrated test confirmed that a second attempt for the same person on the same day displayed an 'Already Marked Today' message.

The check should handle an empty file and a file containing only a header. It should also account for missing or malformed files in a robust implementation. If multiple processes can write at once, a CSV-based check can still race; a database with a unique constraint would be safer.

Logic in plain language

- Read the existing attendance rows.
- Get today's date from the system clock.
- Search for a row with the same name and today's date.
- If found, do not append another row.
- If not found, append name, date and time.

15. APPLICATION WORKFLOW AND USER OPERATION

Before starting, the user should confirm that the virtual environment is active and that the webcam is available. The application is launched from the project root with the command python app.py. The application trains the recognizer, opens the camera and begins processing frames.

The user positions their face in view of the camera and waits for detection and recognition. If the identity is accepted and attendance has not already been recorded for that date, the program writes the CSV entry. If a row already exists, the duplicate-prevention message is shown instead.

The webcam session can be closed using the application's configured quit key. In the demonstrated workflow, pressing Q closed the camera window. The capture object should be released and OpenCV windows should be destroyed when the loop ends.

Safe operating practice

Use the application only with informed participants. Explain what is captured, where the images and records are stored, who can access them and when they will be deleted.

16. IMPLEMENTATION STRUCTURE AND KEY CODE PATTERNS

The project separates concerns conceptually even if the prototype is implemented in one main Python file. Model setup loads the classifier and trains the recognizer. The camera loop handles frame capture and display. The attendance helper checks the CSV and writes new entries.

Keeping these responsibilities in separate functions improves readability and makes later testing easier. For example, a function for duplicate checking can be tested with a temporary CSV file without opening the webcam. A separate image-loading function can be tested using a small sample dataset.

The current project also includes a README, requirements file, attendance CSV, face cascade XML file and a known_faces directory. A .gitignore file is used to exclude local environment and private data from Git tracking.

Maintainability suggestions

- Use descriptive function names and constants for paths and thresholds.
- Validate file and camera operations and show useful error messages.
- Avoid duplicating CSV-reading logic in multiple places.
- Keep personal images and attendance records out of public commits.
- Document setup commands in README.md.

17. INSTALLATION AND SETUP GUIDE

Install Python 3.12 and verify it with the Python launcher. Open the project folder in Visual Studio Code and create a virtual environment using py -3.12 -m venv .venv. Activate it in PowerShell with .\.venv\Scripts\Activate.ps1.

Upgrade pip and install dependencies from the requirements file. The project environment used OpenCV contrib, NumPy, Flask, pandas, openpyxl and Pillow. The exact dependency list should be checked in requirements.txt before reproducing the setup.

Verify the OpenCV face module by importing cv2 and checking that cv2.face.LBPHFaceRecognizer_create exists. Confirm that the Haar Cascade XML file is present at the path used by app.py. Then run python app.py from the project root.

Typical PowerShell commands

- `py -3.12 --version`
- `py -3.12 -m venv .venv`
- `.\.venv\Scripts\Activate.ps1`
- `python -m pip install --upgrade pip`
- `pip install -r requirements.txt`
- `python app.py`

18. GIT AND PROJECT VERSION CONTROL

Git records source-code changes as commits. The local repository was configured with a user name and email address, then a commit was created for the attendance-system update. The project was pushed to the GitHub repository after authentication through Visual Studio Code.

Git identity configuration is metadata used to label commits; it does not itself sign in to GitHub. The user name and email can be set globally or only for the current repository. The email should be one the user is comfortable associating with public commits.

Because the repository is public, sensitive data must not be committed. The known_faces directory contains biometric images and attendance.csv contains personal attendance information. A .gitignore rule should exclude these private files while retaining a placeholder file if the folder structure needs to be visible.

Recommended .gitignore entries

- `.venv/`
- `__pycache__/`
- `known_faces/*`
- `!known_faces/.gitkeep`
- `attendance.csv`

19. TESTING STRATEGY AND TEST CASES

Testing was performed manually during development. The aim was to verify that the application could start, train the model, access the webcam, recognize the enrolled user and update the attendance file. A second attempt on the same day was used to check duplicate prevention.

The test cases below describe functional checks. They do not represent a formal benchmark across many people or environments. For a stronger evaluation, collect a separate test set, include unknown participants, vary lighting and pose, and record false acceptances and false rejections.

Each test should be repeated after major changes to the dataset, model parameters, OpenCV version or camera configuration. Test records should use synthetic or consented data and should not expose private biometric images.

Representative test cases

- TC-01: Start application with valid dependencies — expected: training starts and completes.
- TC-02: Start with webcam available — expected: camera window opens and frames are processed.
- TC-03: Present enrolled face — expected: application predicts the enrolled identity in demonstration conditions.
- TC-04: First attendance attempt of the day — expected: one row is appended.
- TC-05: Repeat same identity on same date — expected: no duplicate row; duplicate message appears.
- TC-06: Press Q — expected: webcam window closes and camera resource is released.

20. OBSERVED DEMONSTRATION RESULTS

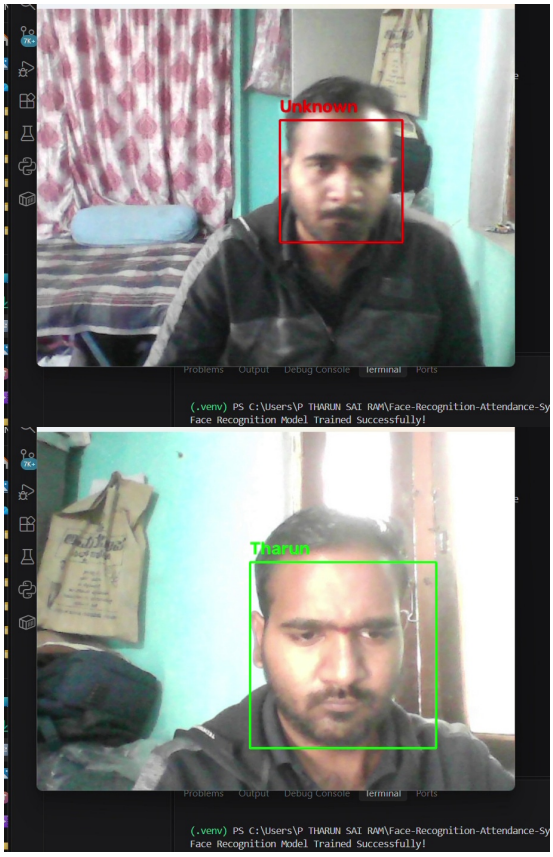
The recorded demonstration showed the model training successfully and the attendance application starting. The webcam recognized the enrolled user, and the terminal displayed a message indicating that attendance was marked. The attendance CSV contained the expected three-column structure: Name, Date and Time.

A subsequent run displayed the 'Already Marked Today' message for the same identity and date. This confirms the duplicate-prevention behavior in the tested scenario.

The screen recording and development screenshots are included as supporting evidence. These observations demonstrate a working prototype under the recorded conditions. They do not establish accuracy for other people, cameras, rooms or lighting conditions.

Evidence included

The report contains two images taken from the demonstration material. They are illustrative records of the interface and workflow; they should not be interpreted as controlled scientific measurements.



Screenshots extracted from the supplied screen-recording/report evidence.

21. PERFORMANCE CONSIDERATIONS

Processing speed depends on frame dimensions, camera frame rate, CPU capability, number of detected faces, cascade parameters and the size of the training dataset. A larger frame can improve visual detail but requires more computation. Reducing frame dimensions can improve responsiveness while potentially making small or distant faces harder to detect.

Training time increases with the number of enrollment images and the cost of face detection on each image. For a small local dataset, retraining at application startup is simple. A larger application may save a trained model and reload it, while ensuring that the saved model corresponds to the current enrollment data.

Performance should be measured rather than guessed. Useful measurements include startup/training time, average processing time per frame, recognition response time and resource use. No numeric speed or accuracy claims are made here because no formal benchmark was recorded.

Practical tuning

Start with a moderate camera resolution and a well-lit scene. Change one parameter at a time and record the effect on missed detections, false detections and responsiveness.

22. SECURITY, PRIVACY AND RESPONSIBLE USE

Face images are biometric information and should be handled carefully. Participants should be informed about the purpose of collection, the storage location, access permissions and retention period. Collect only the minimum images needed for the project and delete them when they are no longer required.

The repository is public, so private face images and attendance records should remain local. The .gitignore file should exclude known_faces contents and attendance.csv. If any sensitive file has already been pushed, adding it to .gitignore does not remove earlier copies from Git history; the repository history and any clones must be considered.

The current application does not include liveness detection. A person may be able to present a printed photograph or an image on a phone to the camera. Therefore, the system should not be used as the sole proof of identity, for payroll, examinations, access control or other high-impact decisions.

Responsible deployment checklist

- Obtain clear consent and provide an alternative attendance method.
- Restrict access to enrollment images and attendance records.
- Set a retention and deletion policy.
- Do not publish biometric images or real attendance logs.
- Provide a way to correct errors and handle recognition failures.
- Evaluate fairness and performance across varied participants and conditions.

23. LIMITATIONS AND RISK ANALYSIS

Lighting changes can alter facial texture and contrast. Side-facing poses, masks, glasses, motion blur and low camera resolution can reduce detection or recognition quality. A Haar Cascade may miss a face or detect a non-face region under difficult conditions.

LBPH is a classical appearance-based method and can be sensitive to image alignment and illumination. Its distance threshold requires local validation. A threshold that accepts a known person reliably may also accept an unknown person if it is too permissive.

CSV storage is simple but lacks database transactions, user authentication and audit controls. The application also relies on the computer's local clock. A wrong system date can result in an incorrect attendance date. The prototype has no administrator dashboard or secure remote backup.

Risk response

Treat the application as a demonstration. Use manual verification for uncertain matches, retain an alternative attendance method and do not make consequential decisions from a single automated prediction.

24. TROUBLESHOOTING GUIDE

If Python cannot find a package, confirm that the terminal shows (.venv) and install requirements inside that environment. If the wrong Python interpreter is selected in VS Code, choose the interpreter located inside the project's .venv folder.

If cv2.face is missing, check that the contrib OpenCV package is installed in the active environment and avoid installing conflicting opencv-python and opencv-contrib-python packages together. Verify the Haar Cascade XML path if the classifier fails to load.

If the webcam does not open, close other camera applications, check Windows camera permissions and verify the camera index. If attendance is not written, confirm that the project directory is writable and that the CSV header matches the three fields written by the application.

Common checks

- Run python --version and python -m pip show opencv-contrib-python.
- Check the interpreter selected by VS Code.
- Confirm haarcascade_frontalface_default.xml exists in the expected folder.
- Close video-call applications before opening the webcam.
- Open attendance.csv and verify the header is Name,Date,Time.
- Check terminal messages before changing code.

25. FUTURE ENHANCEMENTS

A web dashboard can display attendance records in a browser using Flask. A dashboard should initially be read-only and should not expose biometric images. It can show totals, dates and a table of attendance entries while preserving the existing webcam application as a separate process.

SQLite or another database can replace CSV when multiple users or simultaneous writes are required. A database can enforce a unique constraint on person and date, support queries and reduce file-format problems. Export to CSV or Excel can remain available for reporting.

Further enhancements include a student or employee registration interface, configurable recognition thresholds, liveness checks, administrator authentication, encrypted storage, audit logs, backup and restore, multiple camera support and a formal evaluation dataset. Each enhancement should be tested independently.

Suggested development roadmap

- Phase 1: improve input validation and error handling.
- Phase 2: build a read-only Flask attendance dashboard.
- Phase 3: migrate attendance storage to SQLite.
- Phase 4: add secure enrollment and administration features.
- Phase 5: evaluate liveness, privacy and accuracy before any real deployment.

26. CONCLUSION AND LEARNING OUTCOMES

The Face Recognition Based Attendance Management System demonstrates an end-to-end computer-vision workflow using Python and OpenCV. It combines local image enrollment, Haar Cascade face detection, LBPH recognition, webcam processing and CSV attendance recording.

The working demonstration confirmed that a registered face could be recognized in the tested environment and that attendance could be recorded with a date and time. The repeat test also confirmed that the same person was not recorded twice on the same day.

Through this project, practical experience was gained in Python environment setup, package management, image-processing concepts, camera handling, file operations, debugging, Git configuration and publishing a code repository. The prototype provides a foundation for further development while its privacy, accuracy and spoofing limitations remain important.

Final note

The project should be described honestly as a learning prototype. Any future real-world use would require consent, security controls, stronger validation, an alternative attendance process and an assessment of applicable laws and policies.

27. APPENDIX A — PROJECT FOLDER AND COMMANDS

The local project folder contains the Python application, dependency list, attendance data file, Haar Cascade XML classifier, README documentation and the known_faces enrollment directory. A virtual environment is stored in .venv and should not be committed to the public repository.

Expected structure (the contents of known_faces and attendance.csv should remain private):

Folder layout

- `Face-Recognition-Attendance-System/`
- ■■■ `app.py`
- ■■■ `requirements.txt`
- ■■■ `README.md`
- ■■■ `.gitignore`
- ■■■ `haarcascade_frontalface_default.xml`
- ■■■ `attendance.csv (local/private)`
- ■■■ `known_faces/Tharun/ (local/private)`
- ■■■ `.venv/ (local environment)`

Useful commands

- `git status` — inspect changed files.
- `git add app.py README.md requirements.txt .gitignore` — stage selected source files.
- `git commit -m "Describe the update"` — save a commit.
- `git push origin main` — publish commits to the remote repository.

APPENDIX B — QUICK REFERENCE, GLOSSARY AND REFERENCES

Illustrative code patterns (adapt to the exact functions in app.py):

```
import cv2
cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
if cascade.empty():
  raise RuntimeError('Could not load face cascade')

camera = cv2.VideoCapture(0)
if not camera.isOpened():
  raise RuntimeError('Camera unavailable')
# Read and validate frames in a loop.
# Release camera when finished.
```

CSV format: `Name,Date,Time` — one row per accepted attendance event.

Glossary

Attendance record: stored presence entry with date and time. **Biometric data:** information derived from a physical characteristic such as a face image. **CSV:** comma-separated values. **Face detection:** locating a face region. **Face recognition:** comparing a face against enrolled identities. **Haar Cascade:** cascade classifier using Haar-like features. **LBPH:** Local Binary Patterns Histograms. **False acceptance:** unknown person accepted. **False rejection:** known person rejected.

References

OpenCV documentation — https://docs.opencv.org/ | Python documentation — https://docs.python.org/3/ | Git documentation — https://git-scm.com/doc | NumPy documentation — https://numpy.org/doc/ | Project repository — https://github.com/pendyalatharun14-a11y/Face-Recognition-Attendance-System

Report note: This is personal project documentation. Code snippets are explanatory; consult the current repository for the exact implementation. No institutional approval or formal accuracy benchmark is claimed.